

Evaluación Experimental de BSTs Autoajustables bajo Secuencias de Acceso No Estacionarias

Sebastián Nieto Paz, Matías Meneses Roncal

Resumen—Los árboles binarios de búsqueda autoajustables reorganizan su estructura para adaptarse a patrones de acceso con localidad. Si bien sus propiedades teóricas están bien establecidas, su evaluación experimental se ha concentrado en workloads estáticos. Este trabajo presenta un estudio del comportamiento adaptativo de Splay Trees, Tango Trees y Multi-Splay Trees, con Red-Black Tree como baseline estático, frente a secuencias con transiciones de fase controladas. Se generaron 20 familias de transición combinatoria entre cuatro propiedades de búsqueda fundamentales, además de un dataset real NASA-HTTP. Los resultados se evaluaron mediante costo amortizado, costo de adaptación, costo de recuperación y costo neto acumulado.

Index Terms—BST, Splay Trees, Tango Trees, Multi-Splay Trees, dynamic optimality, locality, workloads dinámicos.

1. INTRODUCCIÓN

LOS árboles binarios de búsqueda autoajustables reorganizan su estructura durante las operaciones para explorar patrones de acceso con localidad temporal o espacial. Los Splay Trees [?] son los más conocidos por sus propiedades amortizadas y su relación con la conjetura de Dynamic Optimality. Los Tango Trees [?] y Multi-Splay Trees [?] ofrecen garantías $O(\log \log n)$ -competitivas respecto al óptimo offline.

Sin embargo, las evaluaciones experimentales existentes [?] se concentran en workloads estáticos. En escenarios reales, los patrones evolucionan: el working set se desplaza, la localidad se degrada y las fases de acceso transicionan entre distintos regímenes. La pregunta central es: ¿cómo se comportan distintos BSTs autoajustables frente a cambios dinámicos de localidad? Presentamos una evaluación experimental sistemática de BSTs autoajustables bajo secuencias con transiciones controladas entre cuatro propiedades de búsqueda —secuencial (S), working set (W), localidad espacial (F) y aleatorio (R)— donde cada familia genera secuencias que enfatizan una propiedad clásica de acceso. La adaptación se evalúa mediante costo amortizado por acceso, costo de adaptación, costo de recuperación y costo neto acumulado.

2. METODOLOGÍA

Se implementaron cuatro familias estacionarias de referencia siguiendo a Al-Adhami & Chheda [?]: secuencial ($1, 2, \dots, n$ repetido), aleatorio uniforme, working set (subconjunto accedido con mayor frecuencia) y localidad espacial (caminata aleatoria con radio acotado). Cada familia enfatiza una propiedad clásica de acceso y sirve como referencia estacionaria para calcular Δ_{adapt} . Sobre estas se construyeron 20 familias de transición: 8 pares dirigidos ($P_1 \rightarrow P_2$: $S \rightarrow W$, $S \rightarrow F$, $W \rightarrow S$, $W \rightarrow F$, $W \rightarrow R$, $F \rightarrow S$, $F \rightarrow W$, $R \rightarrow W$) y 12 triples —6 de retorno ($A \rightarrow B \rightarrow A$)

y 6 de cadena ($A \rightarrow B \rightarrow C$, correspondientes a las 6 permutaciones de $\{S, W, F\}$)—, excluyendo el caso estacionario ($P \rightarrow P$) utilizado únicamente como referencia estacionaria. R participa solo en $W \rightarrow R$ y $R \rightarrow W$ porque el acceso aleatorio anula toda estructura previa, haciendo no informativos los pares restantes con R. Cada traza tiene $n = \{1023, 8191, 32767\}$ claves y longitud de fase $5n$, con radio de working set y localidad espacial $k = 64$. Cada traza se generó con 2 semillas independientes (seed 2026 y 2027) para capturar variabilidad; las métricas reportadas en los gráficos promedian ambas réplicas.

Se integró el dataset NASA-HTTP (Kennedy Space Center, Julio-Agosto 1995, $\sim 3.4\text{M}$ accesos) con URLs transformadas a claves mediante mapeo alfabético de paths, de modo que archivos del mismo directorio reciban claves cercanas y se preserve la estructura jerárquica del sitio web; lo que permite evaluar fluctuaciones naturales de un dataset real.

Los BSTs se integraron desde `adhmi3310/tango` (MIT), añadiendo un Red-Black Tree [?] como baseline sin reorganización durante búsqueda. Cada estructura expone un contador de operaciones que acumula recorridos de puntero y rotaciones por acceso, compatible con el modelo BST de Wilber [?].¹

Se definieron cuatro métricas.

- Costo amortizado por acceso: promedio de operaciones elementales (recorridos de puntero y rotaciones) por búsqueda.
- Costo de adaptación: $\Delta_{\text{adapt}} = \overline{\text{ops}}_{\text{trans}}(P_2) - \overline{\text{ops}}_{\text{est}}(P_2)$, donde $\overline{\text{ops}}_{\text{trans}}(P_2)$ es el promedio de toda la fase P_2 en la traza de transición y $\overline{\text{ops}}_{\text{est}}(P_2)$ es el promedio de la traza estacionaria pura de P_2 .
- Costo de recuperación: $\Delta_{\text{recup}} = \overline{\text{ops}}_{\text{fase 3}} - \overline{\text{ops}}_{\text{fase 1}}$, promediando cada fase completa.

1. Código y datos disponibles en <https://github.com/flauts/EDA-Project>.

- Costo neto acumulado: mismo cálculo que Δ_{recup} aplicado a triples de cadena ($A \neq C$).

3. RESULTADOS

3.1. Escalamiento en Regímenes Estacionarios

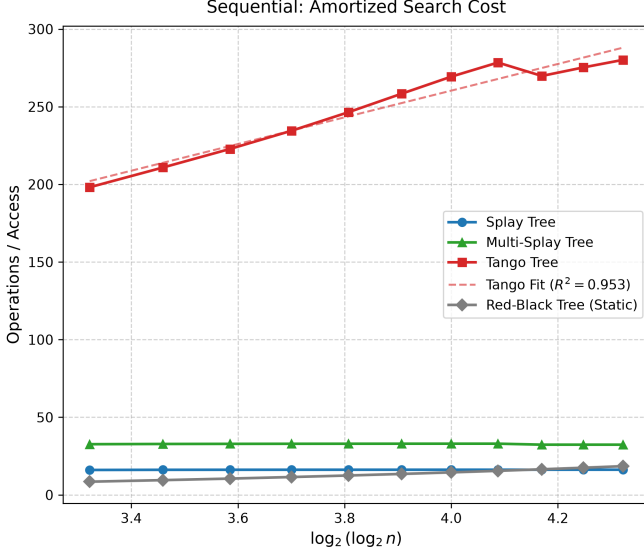


Figura 1. Costo amortizado por acceso vs. n en escala $\log \log$ para acceso secuencial. Splay Tree mantiene costo $O(1)$ constante por acceso, mientras que Tango y Multi-Splay presentan costos superiores debido a su estructura auxiliar.

La Figura ?? muestra el escalamiento en acceso secuencial (S). Splay Tree presenta costo amortizado $O(1)$ por acceso como predice la teoría, mientras que Tango Tree muestra un costo significativamente mayor debido a la sobrecarga de mantener caminos preferidos en árboles auxiliares. Multi-Splay se ubica entre ambos.

3.2. Adaptación entre Pares: Momentum Estructural

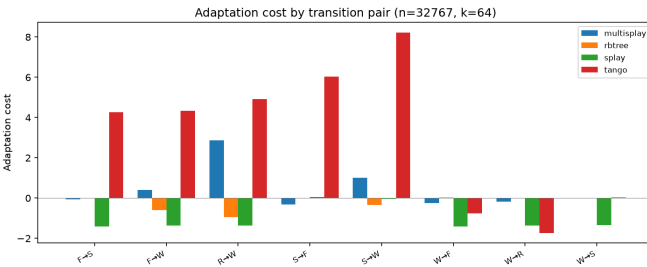


Figura 2. Costo de adaptación Δ_{adapt} para los 8 pares de transición con $n = 32767$, $k = 64$ (promediado sobre 2 semillas). Valores negativos indican que la estructura heredada de P_1 favorece el desempeño en P_2 .

La Figura ?? muestra el costo de adaptación para cada par dirigido ($P_1 \rightarrow P_2$). Splay Tree exhibe costos negativos en la mayoría de las transiciones ($-1,3$ a $-1,4$ ops/acceso), excepto en aquellas que parten de acceso secuencial ($S \rightarrow F$ y $S \rightarrow W$), donde el costo es cercano a cero. Esto sugiere que la estructura heredada de una fase previa distinta a la

secuencial beneficia el desempeño en la fase siguiente, efecto que denominamos *momentum estructural*.

Tango Tree presenta Δ_{adapt} moderadamente positivo en la mayoría de las transiciones ($+4$ a $+8$ ops/acceso), reflejando la sensibilidad de su estructura de caminos preferidos al cambio de patrón. En las transiciones que parten de working set ($W \rightarrow F$, $W \rightarrow R$, $W \rightarrow S$) el costo es cercano a cero, lo que sugiere que la estructura de preferred paths acumulada durante el working set no perjudica la siguiente fase. Multi-Splay alterna entre valores cercanos a cero y positivos moderados ($+1$ a $+3$ ops/acceso), con la mayor penalización en $R \rightarrow W$, consistente con la dificultad de reorganizarse tras un patrón aleatorio. El Red-Black Tree permanece invariante, consistente con su naturaleza estática.

3.3. Resiliencia y Acumulación en Transiciones Múltiples

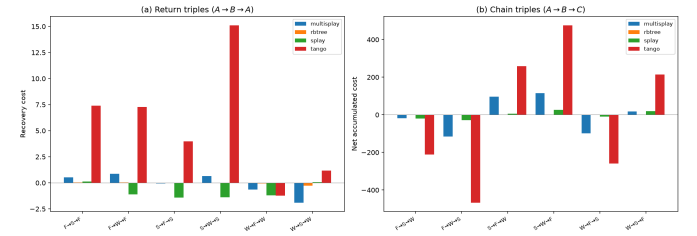


Figura 3. (a) Costo de recuperación Δ_{recup} para triples de retorno ($A \rightarrow B \rightarrow A$). (b) Costo neto acumulado para triples de cadena ($A \rightarrow B \rightarrow C$, $A \neq C$). Promediado sobre 2 semillas y todos los n ($k = 64$).

Las secuencias de retorno ($A \rightarrow B \rightarrow A$) en la Figura ??(a) miden la resiliencia estructural: la capacidad del árbol de recuperar su rendimiento original tras ser perturbado por una fase intermedia. Splay Tree recupera con Δ_{recup} cercano a cero, lo que sugiere que su estructura se reconfigura al patrón original. Tango presenta valores positivos en todas las configuraciones ($+3$ a $+14$ ops/acceso), indicando que la fase intermedia deja una huella persistente en sus caminos preferidos. Multi-Splay se ubica en un rango intermedio ($< 2,5$). El Red-Black Tree permanece invariante.

Las cadenas ($A \rightarrow B \rightarrow C$) en la Figura ??(b) capturan el efecto acumulado de dos transiciones consecutivas. Splay Tree preserva su ventaja en las 6 permutaciones, con costos netos cercanos a cero o negativos, mostrando que su costo no aumenta tras múltiples transiciones. Tango acumula penalizaciones en cada salto, Multi-Splay mantiene una posición intermedia, y el Red-Black Tree refleja la diferencia intrínseca entre los patrones A y C .

3.4. Validación con Dataset Real NASA-HTTP

La Figura ?? muestra la comparación sobre ambas trazas NASA (Julio y Agosto 1995). La secuencia de Julio exhibe una fuerte localidad temporal: el 1 % de las claves concentra el 80 % de los accesos, con un intervalo mediano de 74 accesos entre repeticiones de una misma clave.

La localidad espacial, en cambio, es escasa: las claves cambian en cada acceso y solo el 16 % de los saltos consecutivos son menores a 100 claves. Aunque el mapeo alfabético agrupa archivos del mismo directorio en claves cercanas,

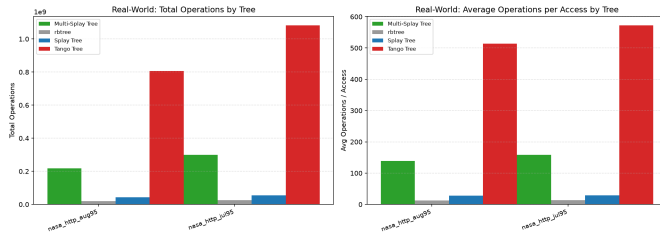


Figura 4. Comparativa del costo amortizado por acceso en las trazas NASA-HTTP (Kennedy Space Center, Julio–Agosto 1995).

el patrón de navegación web salta constantemente entre directorios, por lo que esta proximidad rara vez se traduce en accesos consecutivos a claves vecinas.

En la traza de Julio de 1995 (la de mayor tráfico), Splay Tree procesa cada acceso en 29.13 operaciones en promedio, Multi-Splay en 158.48 ($5.4\times$ peor) y Tango en 572.50 ($19.7\times$ peor). El Red-Black Tree registra 13.65 operaciones —el menor absoluto— porque su búsqueda balanceada sin rotaciones adicionales es suficiente cuando la localidad, aunque presente, no es lo bastante concentrada para justificar el costo de las rotaciones adaptativas de Splay. El ordenamiento es consistente con lo observado en trazas sintéticas y en Agosto de 1995.

4. CONCLUSIONES

Se evaluó experimentalmente el comportamiento de BSTs autoajustables bajo secuencias con transiciones de fase controladas. Los resultados muestran lo siguiente:

Splay Tree fue la estructura más reactiva, con el menor costo de adaptación en todas las transiciones y valores incluso negativos en $F \rightarrow S$ y $F \rightarrow W$. Se observó que ciertas configuraciones generadas durante una fase pueden beneficiar el desempeño en la fase siguiente.

Tango Tree sufre una sobrecarga estructural persistente. Aunque posee una garantía $O(\log \log n)$ -competitiva [?], [?], el diseño de Tango introduce una sobrecarga práctica considerable. En los experimentos, esta sobrecarga se tradujo en costos aproximadamente $20\times$ superiores a Splay en todas las familias de acceso.

Multi-Splay Tree ocupa una posición intermedia, con factores constantes que lo sitúan $4\text{--}6\times$ por debajo de Splay en términos prácticos, reproduciendo la brecha entre optimalidad asintótica y eficiencia práctica observada en Al-Adhami & Chheda [?].

El Red-Black Tree obtuvo el menor costo absoluto observado, pero no se adapta. Su costo (13.65 ops/acceso en NASA-HTTP) es el menor de todos, lo cual sugiere que si el workload carece de localidad explotable, una estructura balanceada estática puede ser suficiente.

En conjunto, los resultados muestran que, dentro de los escenarios evaluados, Splay Tree ofrece la mejor capacidad de adaptación frente a cambios de localidad, aun cuando otras estructuras poseen garantías competitivas más fuertes. Los resultados corresponden a las implementaciones y patrones de acceso considerados.

REFERENCIAS

- [1] D. D. Sleator and R. E. Tarjan, “Self-adjusting binary search trees,” *J. ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [2] E. D. Demaine, D. Harmon, J. Iacono, and M. Pătraşcu, “Dynamic optimality—almost,” *SIAM J. Comput.*, vol. 37, no. 1, pp. 240–251, 2007.
- [3] C. Wang, J. Derryberry, and D. D. Sleator, “ $O(\log \log n)$ -competitive dynamic binary search trees,” in *SODA*, 2006, pp. 374–383.
- [4] K. Al-Adhami and D. Chheda, “Theoretical insights and an experimental comparison of tango trees and multi-splay trees,” *arXiv:2405.18825*, 2024.
- [5] R. Wilber, “Lower bounds for accessing binary search trees with rotations,” *SIAM J. Comput.*, vol. 18, no. 1, pp. 56–67, 1989.
- [6] L. J. Guibas and R. Sedgwick, “A dichromatic framework for balanced trees,” in *Proc. 19th FOCS*, 1978, pp. 8–21.